

California Bureau of Automotive Repair

DAD Communication Specification

Version 4.00

Submitted Date:

October 14, 2013

**Proprietary and CONFIDENTIAL – California Bureau of Automotive Repair
Developed by SGS Testcom Inc.**

Table of Contents

1	INTRODUCTION.....	1
1.1	PURPOSE.....	1
1.2	APPROACH.....	1
1.3	REVISIONS.....	1
2	APPLICATION PROGRAMMING INTERFACE (API) DEFINITION.....	4
2.1	DLL REQUIREMENTS.....	4
2.2	ENUMERATIONS.....	4
2.2.1	<i>CommunicationResult</i>	4
2.2.2	<i>SelfTestResponse</i>	5
2.3	LOGGING.....	5
2.4	PROTOCOL NAMES.....	5
2.5	NON-STANDARD COMMUNICATION LOGIC.....	6
2.6	PROPERTIES.....	7
2.6.1	<i>Port</i>	7
2.6.2	<i>Max Wait Time</i>	7
2.6.3	<i>OBD Protocol</i>	7
2.6.4	<i>Vendor Log</i>	8
2.6.5	<i>Serial Data Log</i>	8
2.7	METHODS	9
2.7.1	<i>Open</i>	9
2.7.2	<i>Close</i>	10
2.7.3	<i>Reset</i>	10
2.7.4	<i>Initialize</i>	10
2.7.5	<i>Get Firmware Version</i>	12
2.7.6	<i>Get Serial Number</i>	12
2.7.7	<i>Self Test</i>	13
2.7.8	<i>Clear Logs</i>	13
2.7.9	<i>Get Voltage</i>	14
2.7.10	<i>Get Mode PID</i>	14
2.7.11	<i>Get Multiple PIDs</i>	17
3	UML.....	18

1 Introduction

1.1 Purpose

The purpose of this document is to define the Application Programming Interface (API) that will be used by the BAR's OBD Inspection System Software (BAR-OIS Software) to communicate with the OBDII Data Acquisition Device. The API will define all required methods that any BAR Certified DAD Vendor must implement in order to interact with the BAR-OIS Software.

1.2 Approach

The approach taken with this API:

- Provides a limited number of methods to be implemented while providing the greatest flexibility for obtaining OBDII data from the vehicle.
- Limits as much as possible, the amount of logic and functionality provided by the DAD software and maximize as much as possible the logic and functionality within the BAR-OIS Software. Centralizing the core logic in the BAR-OIS Software will enable the BAR to make programmatic changes centrally without requiring coordination of changes with multiple BAR Certified DAD Vendors.
- Creates a communication transport and communication protocol agnostic solution. The BAR Certified DAD Vendors are free to communicate with the OBDII DLC over USB, Bluetooth or any other available or future technology without requiring any change to the BAR-OIS Software.
- Expects that when an API method is called:
 - the method implementation will handle all communication, timing, and multi-frame responses atomically.
 - the method will provide a single response back to the caller containing all possible data returned in response to the request.
 - the method will respond with the raw, unmodified OBDII data unless otherwise specified in this document.

1.3 Revisions

Revision Number	Revision Description	Revision Date
0.01	Original Draft, submitted to BAR for review.	May 18, 2012
0.02	Updated specification and submitted to BAR with track changes for review.	June 19, 2012
0.03	Accepted changes from V0.01 and formatting changes. Added serial data log and edited communication results, sent to BAR for review. Submitted to BAR for review.	June 22, 2012
0.04	Added changes from OBD requirements meetings with BAR	September 11, 2012
1.0	Accepted modifications. Responded to BAR's change requests. Added log clearing handling	November 1, 2012

Revision Number	Revision Description	Revision Date
1.01	Updated API to include a Clear Logs method. Updated GetModePID method description	November 30, 2012
1.02	Removed specific formatting requirements for Mode 6 data responses.	December 3, 2012
1.03	Updated Serial Data Log property with a description of the required format	January 4, 2013
1.04	Changed NegativeResponseDelay to MaxWaitTime for clarity. Note that this version of the document corresponds with version 0.19.196 of IDAD.dll	January 31, 2013
1.05	Small phrasing changes. Fixed the section numbering.	February 6, 2013
1.06	Clarification of Reset and Close. Amending the Communication Result table to include additional NRCs.	February 11, 2013
1.07	Fixed a typo. Updated the description of the reset, close, and initialize methods.	March 4, 2013
1.08	Amending descriptions for Reset, Close, GetVoltage, and Protocol Names. Adding an Open method. Removing the OBDProtocolAttemptOrder property and adding it as an argument for the Initialize method. A VIN mask has also been added as an argument for the Initialize method. The methods have been reordered to have Open and Close first to emphasize their importance in opening and closing communication with the DAD.	April 2, 2013
1.09	Updated descriptions for Clear Logs method, Serial Log property, and Vendor Log Property. Updated GetVoltage description to match the requirements for resolution and accuracy. Clarified OBD Protocol property description to denote who is setting the property.	May 5, 2013
1.10	Updated Mode/PID examples	June 6, 2013
1.11	Fixed the title in a Mode/PID VIN example	June 11, 2013
1.12	Updated the Initialize description and OBD Protocol property description. Added a new section for Non-Standard Communication Logic	July 3, 2013
1.13	Further edits to the Initialize method description. Updated the Self Test method description as well.	July 17, 2013
2.00	Accepting all changes from V1.13 and updating version	July 17, 2013
2.01	Adding a revision description for V2.00. Corrected typos in the Initialize description. Clarified the sending and receiving formats for protocol names. Clarified how the DAD should handle unrecognized protocol names.	July 22, 2013
3.00	Accepting all changes from V2.01	July 23, 2013

Revision Number	Revision Description	Revision Date
3.01	Adding information and examples on the Mode 06 response format	October 10, 2013
4.00	Accepted 3.01 changes	October 14, 2013

2 Application Programming Interface (API) Definition

2.1 DLL Requirements

The API shall be implemented and delivered as a DLL which can be invoked by a Microsoft .NET 4.0 application. The DLL itself should communicate with the BAR-OIS Software and the DAD in the most robust way possible. The DLL shall also do any negotiations necessary to determine the OBDII protocol the vehicle uses and which vehicle module reports the appropriate response and that shall be transparent to the user of the DLL.

All members described in the document are part of the IDAD interface provided by SGS Testcom for a BAR Certified DAD Vendor to implement. The assembly containing the IDAD interface and objects described in Section 3 will be provided to potential BAR Certified DAD Vendors. Any method should be able to be called at any time with exception of Initialize and Close, provided each method is called after the last one has returned. Reset should be able to be called at any point handling whatever state the interface is currently in.

The .NET assembly provided by the BAR Certified DAD Vendors shall be strongly named and signed in accordance with the standard .NET paradigm: <http://msdn.microsoft.com/en-us/library/xwb8f617.aspx>. Any updates to the BAR Certified DAD Vendor's assembly will need to be coordinated with the BAR's BAR-OIS Software Contractor so that the overall security of this interface's implementations remains intact and no unauthorized implementations are used.

2.2 Enumerations

This section lists the enumerations to be used in the public methods of the DLL. These are intended to keep the code readable. Where important, a type and values for the enumeration members are specified.

2.2.1 CommunicationResult

The CommunicationResult enumeration is part of the result value for all public methods. The values are as follows:

Name	Description
Success	Communication was successful. The command was successfully sent to the vehicle, and, if applicable, a valid response was received back from the vehicle.
DADTimedOut	Communication timed out waiting for a response from the DAD.
VehicleTimedOut	Communication timed out waiting for a response from the vehicle.
DADError	Error communicating with the DAD not otherwise described.
VehicleError	Error communicating with the vehicle not otherwise described.
ErrorOpeningPort	There was an error when opening the port on the computer for communication with the DAD.
PortInUse	The port on the Computer was already in use when trying to start communication with the DAD.
DADNotConnected	The DAD is not connected to the Computer.
VehicleNotConnected	The vehicle is not connected to the DAD.
NotSupported	The command issued is not supported by the DAD, such as a Mode/PID combination. This response does not necessarily mean the vehicle does not support this command.
InvalidData	The data returned by the vehicle appears mangled in some way (e.g. the checksums do not match), but is still returned and passed back to the BAR-OIS Software. Note: The DAD is not expected to validate the OBDII data to the J1979 SAE Standards.

2.2.2 SelfTestResponse

The SelfTestResponse enumeration contains the result of any self test the DAD may be requested to perform, its members are as shown below, and may be added to at DAD Vendor's suggestion:

Name	Description
Passed	Self test passed.
UnknownFailure	Self test failed for unknown reason.
CableFailure	Self test failed due to a failure in the cable.
Failed	Self test failed for a known reason not otherwise described.

2.3 Logging

The DAD will store all log information in memory. The naming, encryption, and storage of the log data will be managed by the BAR-OIS Software.

Log data should only be cleared when the Clear Log method has been called by the BAR-OIS Software. This provides the BAR-OIS Software control over the timeframe for which these logs are collected.

2.4 Protocol Names

When receiving a protocol name from the BAR-OIS Software, the following protocol names will be used:

Protocol	String Format
J1850 PWM	“JPWM1850”
J1850 VPW	“JVPW1850”
ISO 9141	“I914”
ISO 14230 slow (5 baud)	“KWPS”
ISO 14230 fast (10,400 baud)	“KWPF”
ISO 15765 11 bit header 500 kBit/s	“ICAN11bt500”
ISO 15765 29 bit header 500 kBit/s	“ICAN29bt500”
ISO 15765 11 bit header 250 kBit/s	“ICAN11bt250”
ISO 15765 29 bit header 250 kBit/s	“ICAN29bt250”

When sending a protocol name to the BAR-OIS Software, the following formats should be used:

Protocol	String Format
J1850 PWM	“JPWM1850”
J1850 VPW	“JVPW1850”
ISO 9141	“I914” followed by the Hex keywords received from the vehicle during initialization (i.e., “I9140808” or “I9149494”
ISO 14230 slow (5 baud)	“KWPS” followed by the Hex keywords (key byte 2 then key byte 1) received from the vehicle during initialization without a space (e.g., “KWPS8FE9”)
ISO 14230 fast (10,400 baud)	“KWPF” followed by the Hex keywords (key byte 2 then key byte 1) received from the vehicle during initialization without a space (e.g., “KWPF8F6B”)
ISO 15765 11 bit header 500 kBit/s	“ICAN11bt500”
ISO 15765 29 bit header 500 kBit/s	“ICAN29bt500”
ISO 15765 11 bit header 250 kBit/s	“ICAN11bt250”
ISO 15765 29 bit header 250 kBit/s	“ICAN29bt250”

For formats that require key bytes, the asterisk character (*) shall be used as a filler character in vehicles that do not properly report two full key bytes.

2.5 Non-Standard Communication Logic

In some cases, non-standard logic will be required to initialize communication with a vehicle.

If the BAR Certified DAD Vendor uses non-standard logic to communicate with “Problem Vehicles”, the BAR Certified DAD Vendor shall define each unique non-standard logic name in conjunction with the BAR’s BAR-OIS Software Contractor and the BAR.

Any defined non-standard logic names can be sent to the DAD as part of a call to the Initialize method or received from the DAD as part of the OBD Protocol property.

2.6 Properties

This section lists properties on the IDAD interface.

2.6.1 Port

```
string Port { get; set; }
```

This property provides access to the port on the Computer used for communication with the DAD. This may be of the form “COM1” or any other identifier which indicates to the BAR Certified DAD Vendor DLL where to communicate with the DAD on the particular Computer. This property will be set prior to calling Open and will not be changed between calling Open and Close.

2.6.2 Max Wait Time

```
int MaxWaitTime { set; }
```

This property allows the BAR-OIS Software to set the maximum number of milliseconds for the DAD to wait before sending a response to the BAR-OIS Software. In particular, this time should be enforced during the GetModePID and GetMultiplePIDs methods. Once this max wait time has been met, the DAD should respond to the BAR-OIS Software with any data it has collected and the appropriate communication result. If the DAD receives responses from all ECUs specified in the ECU list in less than this time, the method should return immediately. During this max wait time, the DAD should respect all applicable OBD protocol timings per SAE J1979.

2.6.3 OBD Protocol

```
string OBDProtocol { get; }
```

The format of the string returned should conform to the following format: *protocol*; where *protocol* is the name of the protocol currently being used by the DAD to communicate with the vehicle. See section 2.4 Protocol Names for the valid formats for the protocol names.

Ex: ICAN11bt500

If the DAD used non-standard logic to communicate with the vehicle, the string returned should conform to the following format: *protocol-vendor-special*; where *protocol* is as described above, *vendor* is the DAD Vendor’s two character code and *special* is the name for the non-standard logic used to communicate with the vehicle. See section 2.5 Non-Standard Communication Logic for more information on non-standard logic names.

Ex: JPWM1850-BR-special1

The DAD shall set this property when Initialize has completed successfully and shall clear this property when Reset is called.

2.6.4 Vendor Log

```
string VendorLog { get; }
```

This property gets the log as provided by the BAR Certified DAD Vendor for troubleshooting/diagnostic purposes since the last time the Clear Logs method was called (or initialization of the BAR Certified DAD Vendor supplied DLL). The contents of this data may be determined by the BAR Certified DAD Vendor.

This property shall not require Open to be called before being available to the BAR-OIS Software.

2.6.5 Serial Data Log

```
string SerialDataLog { get; }
```

This property gets the log as provided by the BAR Certified DAD Vendor for all communications since the last time the Clear Logs method was called (or initialization of the DAD Vendor supplied DLL).

This property shall not require Open to be called before being available to the BAR-OIS Software.

The contents of this log should contain the data in the following table, with comma delimited columns, and one message per line (CSV). No header line for column names should be included.

Column Name	Description of Data	Value Range of Value Type	Expected Format of Data
TIME	The time in milliseconds referenced from the creation time of the log file.	Resolution of 1 millisecond	ASCII text
MESS_TYPE	The message type for the data line. Was it a message sent to the vehicle or received from the vehicle.	Shall be "RX MSG:" for messages received from the vehicle and "TX MSG:" for messages sent to the vehicle	ASCII text
PROTOCOL	The protocol used for each line of communication.	See table in section 2.4 for the list of acceptable protocol names.	ASCII text
DATA	The data message sent or received from the vehicle's OBDII system. The data shall be ASCII text that represents the raw Hex values sent over the OBDII system.	Hex represented as ASCII	ASCII text

2.7 Methods

This section lists and details the methods to be made publically available from the DLL. As stated previously, the DLL should handle all underlying communication to the DAD including protocol negotiation and always return a response indicative of the communication's success.

Most methods will return an IDADResponse instance to allow access to the communication result along with the data payload.

2.7.1 Open

`CommunicationResult` Open()

Open should establish communication with the DAD. This would entail any port opening or establishing of wireless communications, ensuring the device is powered on and able to receive commands.

Open should return a `CommunicationResult` which indicates the success or failure of the communication. Open should only return Success if the device is ready to talk to a vehicle.

Open returns the following `CommunicationResults`:

Name
Success
DADTimedOut
DADError
ErrorOpeningPort
PortInUse
DADNotConnected

2.7.2 Close

`void Close()`

The Close method is called when the BAR-OIS Software is finished talking to the DAD. When Close is called, the DLL should take the appropriate steps to end communication with the DAD; e.g., closing the port. If the DAD is currently talking to a vehicle, Close should also end communication with the vehicle.

This method does not have any return value as the BAR-OIS Software will not retry this command or do anything different if the DAD is unable to successfully close communications.

2.7.3 Reset

`IDADResponse<bool> Reset()`

The Reset method should reset the DAD and DLL such that both are ready to connect to a new vehicle. The OBD Protocol Attempt Order and OBD Protocol should be cleared by this method, but the Vendor and Serial logs should not be cleared by this method.

Along with the communication result, this method should return a bool representing whether or not the DAD and DLL were successfully reset.

This method returns the following CommunicationResults:

Name
Success
DADTimedOut
DADError
ErrorOpeningPort
PortInUse
DADNotConnected

2.7.4 Initialize

`CommunicationResult Initialize(IList<string> obdProtocolAttemptOrder, string vinMask)`

The Initialize method should initialize communications with the vehicle, whatever that may entail to encapsulate the details from the user of the DLL. The result is of type CommunicationResult which indicates the success or failure of the communication.

The VIN mask argument will contain the VIN of the current vehicle with one or more characters masked with the '*' character. The VIN mask is provided to the DAD for logging and other troubleshooting purposes as desired by the DAD vendor.

The OBD protocol attempt order argument gives the order in which OBDII protocols should be attempted for the current vehicle. The list may contain one or more strings where each string is a protocol name, as specified in section 2.4 Protocol Names, or non-standard logic name, as per section 2.5 Non-Standard Communication Logic.

Ex: JVPW1850, special1, JPWM1850, special2

The protocols given to the DAD during initialization are not the same as the string expected back from the OBD Protocol property.

Example:

1. The BAR-OIS Software opens communication with the DAD and requests the DAD's serial number. The DAD reports a serial number of AA123456.
2. Based on the vehicle information gathered previously in the test and the DAD Vendor being used, the BAR-OIS Software will select a set of protocol sequences from the database to use for initialization.
3. The BAR-OIS Software directs the DAD to initialize communication with the vehicle using the first sequence of protocols: JPWM1850, Special2XX, ICAN11bt500
4. The DAD first tries to communicate over JPWM1850, but communication using this protocol fails.
5. The DAD next tries to use the non-standard logic identified by the string "Special2XX". This logic successfully initializes communication with the vehicle, so the DAD stops attempting different protocols and returns Success.
6. The BAR-OIS Software (at some point in time after the last step, but before resetting the DAD) requests the OBD Protocol property.
7. Since the actual protocol the DAD ended up using is ISO 9141, the DAD returns "I9140808-AA-Special2XX". This tells the BAR-OIS Software which protocol the DAD is actually using to communicate with the vehicle, the DAD Vendor, and the non-standard logic name used to obtain successful communication with the vehicle.
8. Once the OBD test is complete, the BAR-OIS Software will store "I9140808-AA-Special2XX" to the database.
9. Later, when the BAR has reviewed the data, the BAR will be able to refine the protocol attempt orders so that when a similar vehicle is tested, the BAR-OIS Software will send "Special2XX" first.

The DAD should use its default protocol attempt order if the given list is null or empty, but the DAD should not fallback onto its default protocol attempt order after running through a given list without successful communication.

The DAD shall skip any non-standard logic names in a protocol attempt order argument that the DAD does not recognize. If the DAD does not recognize any of the non-standard logic names in the list, and there are no protocol names, the DAD should use its default protocol attempt order as if the list was empty or null.

This method returns the following CommunicationResults:

Name
Success
DADNotConnected
DADTimedOut
DADError
VehicleTimedOut
VehicleNotConnected
VehicleError

2.7.5 Get Firmware Version

`IDADResponse<string> GetFirmwareVersion()`

The GetFirmwareVersion method should retrieve the firmware version of the DAD. The result is of the type IDADResponse using a CommunicationResult to indicate success or failure of the communication, and a string for the value of the DAD firmware version. Caching of firmware version information by the DLL is not allowed.

This method returns the following CommunicationResults:

Name
Success
DADError
DADNotConnected
DADTimedOut

2.7.6 Get Serial Number

`IDADResponse<string> GetSerialNumber()`

The GetSerialNumber method should retrieve the serial number of the DAD. The result is of the type IDADResponse using a CommunicationResult to indicate success or failure of the communication, and a string for the value of the DAD serial number. Caching of serial number information by the DLL is not allowed.

This method returns the following CommunicationResults:

Name
Success
DADError
DADNotConnected
DADTimedOut

2.7.7 Self Test

`IDADResponse<SelfTestResult> SelfTest()`

The SelfTest method runs the self-test on the DAD. The result is of the type IDADResponse using a CommunicationResult to indicate success or failure of the communication, and SelfTestResult for the result of the self-test itself. The SelfTestResult should default UnknownFailure, and be set to the appropriate response based on the response from the DAD. At a minimum, the SelfTestResults of 'Passed' and 'Failed' must be implemented by the DAD Vendor. Additional SelfTestResults, such as 'CableFailure', are optional. This method should run any diagnostics on the DAD itself and its firmware that the DAD Vendor deems necessary or helpful for troubleshooting.

This method returns the following CommunicationResults:

Name
Success
DADError
DADNotConnected
DADTimedOut

This method returns the following SelfTestResults:

Name
Passed
UnknownFailure
CableFailure
Failed

2.7.8 Clear Logs

`void ClearLogs()`

The Clear Logs method should clear the in-memory Vendor and Serial logs. The method does not have any parameters or return value as no decisions are based on the result of this action.

This method can be called by the BAR-OIS Software before Open has been called. In this case, it must still be able to clear the log files.

2.7.9 Get Voltage

`IDADResponse<decimal> GetVoltage()`

The GetVoltage method should return back the voltage on pin 16 of the OBD connector as measured by the DAD. The result is of type `IDADResponse<decimal>` using a `CommunicationResult` to indicate the success or failure of the communication and a decimal to indicate voltage. The voltage value must be 0.0V to +18.0V with 0.1V resolution and 3% accuracy.

This method returns the following `CommunicationResults`:

Name
Success
DADError
VehicleNotConnected
DADNotConnected
DADTimedOut

2.7.10 Get Mode PID

`IModePIDResponse GetModePID(byte mode, byte pid)`

`IModePIDResponse GetModePID(byte mode, byte pid, IList<byte[]> ecus)`

The GetModePID methods request individual Mode/PID OBD values. The PID argument should be ignored for those OBDII Modes that do not require a PID value. At the request of a Mode/PID combination, the method will return an `IModePIDResponse` to express the communication result along with a list of data responses from all responding ECUs.

The DLL must collect responses from all responding ECUs for the Mode and PID combination request. For each ECU, the DLL will store the Mode, PID, ECU address and the data returned by that ECU in an `IECUData` structure. The ECU addresses shall be represented by one or two bytes as specified in the *BAR OBD Inspection System Data Acquisition Device Specification*.

For the Mode, the DAD will return the request SID that corresponds to the response SID indicated in the ECU response; e.g., \$01 should be returned for the response SID \$41. If the response SID does not correspond to any of Modes \$01 - \$0A, the DAD should return \$00 for the Mode and `InvalidData` for the `CommunicationResult`.

For the method signature without an input list of ECUs, the DAD will wait for responses from all ECUs. For the method signature with an input list of ECU addresses, the DAD will wait for

responses from the given ECUs and any additional ECU that responds before complete responses from the given ECUs are collected. In both cases, the DAD will not wait for longer than the current max wait time for complete responses from any ECU.

When the GetModePID method is called, the calling application will block and wait for the DLL to respond to the request. The DLL will handle multiple frame responses by concatenating them together, removing header and footer bytes which do not pertain to the data so that the returned data appears as if it were all returned in one frame.

In situations where a single ECU may respond with multiple data elements (such as for CVN or CAL ID), the DAD will correctly order and concatenate them before sending them as the data payload through this API.

In the following Mode \$09 examples “header” and “footer” bytes are shown in red:

ISO9141 VIN Message Count = 5

RX MSG: 48 6B 11 49 01 05 13

ECU Address = 11

Mode = 09

PID = 01

Data Payload = 05

ISO9141 VIN = SCCPC11176HL30004

RX MSG: 48 6B 11 49 02 01 00 00 00 53 62

RX MSG: 48 6B 11 49 02 02 43 43 50 43 29

RX MSG: 48 6B 11 49 02 03 31 31 31 37 DB

RX MSG: 48 6B 11 49 02 04 36 48 4C 33 0F

RX MSG: 48 6B 11 49 02 05 30 30 30 34 D7

ECU Address = 11

Mode = 09

PID = 02

Data Payload = 00 00 00 53 43 43 50 43 31 31 31 37 36 48 4C 33 30 30 30 34

J1850VPW VIN = 5GTDN13616810000

RX MSG: 48 6B 10 49 02 01 00 00 00 35 FCS

RX MSG: 48 6B 10 49 02 02 47 54 44 4E FCS

RX MSG: 48 6B 10 49 02 03 31 33 36 31 FCS

RX MSG: 48 6B 10 49 02 04 36 38 31 30 FCS

RX MSG: 48 6B 10 49 02 05 30 30 30 36 FCS

Mode = 09

PID = 02

Data Payload = 00 00 00 35 47 54 44 4E 31 33 36 31 36 38 31 30 30 30 30 36

J1850PWM VIN = YR44E72TA31978

RX MSG: 01 6B 10 49 02 01 31 46 54 59 FCS

RX MSG: 01 6B 10 49 02 02 52 34 34 45 FCS

RX MSG: 01 6B 10 49 02 03 37 32 54 41 FCS

RX MSG: 01 6B 10 49 02 04 33 31 39 37 FCS

RX MSG: 01 6B 10 49 02 05 38 00 00 00 FCS

Mode = 09

PID = 02

Data Payload = 31 46 54 59 52 34 34 45 37 32 54 41 33 31 39 37 38 00 00 00

ISO14230 VIN = 5GZCZ33D86S800124

RX MSG: 87 F1 11 49 02 01 00 00 00 35 (0A)
RX MSG: 87 F1 11 49 02 02 47 5A 43 5A (14)
RX MSG: 87 F1 11 49 02 03 33 33 44 38 (B9)
RX MSG: 87 F1 11 49 02 04 36 53 38 30 (C9)
RX MSG: 87 F1 11 49 02 05 30 31 32 34 (A0)

Mode = 09

PID = 02

Data Payload = 00 00 00 35 47 5A 43 5A 33 33 44 38 36 53 38 30 30 31 32 34

ISO15765 VIN = JM1NC2PF0C0223226

RX MSG: 7E8 10 14 49 02 01 4A 4D 31
RX MSG: 7E8 21 4E 43 32 50 46 30 43
RX MSG: 7E8 22 30 32 32 33 32 32 36

ECU Address = 07 E8

Mode = 09

PID = 02

Data Payload = 01 4A 4D 31 4E 43 32 50 46 30 43 30 32 32 33 32 32 36

Note: For ISO15765, the item count byte is returned as part of the data payload.

For Mode \$06, the TID and MID values that repeat throughout the responses should not be stripped. In the following Mode \$06 examples “header” and “footer” bytes are shown in red:

J1850VPW Mode 06 TID 06

RX MSG: 48 6B 10 46 06 35 00 30 00 6C DD
RX MSG: 48 6B 10 46 06 41 00 2F 00 BC BE
RX MSG: 48 6B 10 46 06 55 00 2E 00 73 E4
RX MSG: 48 6B 10 46 06 61 00 2E 00 D6 B6

Mode = 06

PID = 06

Data Payload = 06 35 00 30 00 6C 06 41 00 2F 00 BC 06 55 00 2E 00 73 06 61 00 2E 00 D6

Note: This differs from Mode 09 responses in that the TID is not stripped from the data payload during concatenation.

ISO15765 Mode 06 MID 02

RX MSG: 7E8 10 1C 46 02 8A 24 01 82
RX MSG: 7E8 21 00 00 04 38 02 8B 0A
RX MSG: 7E8 22 00 00 00 00 00 00 02
RX MSG: 7E8 23 8C 0A 00 00 00 00 00
RX MSG: 7E8 24 00 AA AA AA AA AA AA

Mode = 06

PID = 02

Data Payload = 02 8A 24 01 82 00 00 04 38 02 8B 0A 00 00 00 00 00 02 8C 0A 00 00 00 00 00

Note: This differs from Mode 09 responses in that the MID is not stripped from the data payload during concatenation.

The DAD will handle Negative Response messages by returning the Negative Response Service Identifier, Request Service Identifier, and the Response Code as the data payload. For example, the DAD will return the data payload: 7F 49 78 when it receives a Response Pending negative response to a Mode 09 request.

The DAD will handle timing related logic as a result of a Mode/PID request based on the BAR Certified DAD Vendor’s proprietary logic as how best to handle communication with each

vehicle. Based on the response, a new max wait time may be given to the DAD and the GetModePID request re-issued.

The following Mode/PIDs may be collected using the GetModePID method:

Mode	PIDs
\$01	\$00-\$FF
\$03	N/A
\$04	N/A
\$06	\$00-\$FF
\$07	N/A
\$09	\$00-\$FF
\$0A	N/A

This method returns the following CommunicationResults:

Name
Success
VehicleTimedOut
DADError
VehicleError
DADNotConnected
VehicleNotConnected
NotSupported
InvalidData
DADTimedOut

2.7.11 Get Multiple PIDs

`IModePIDResponse GetMultiplePIDs(byte mode, IList<byte> pids)`

`IModePIDResponse GetMultiplePIDs(byte mode, IList<byte> pids, IList<byte[]> ecus)`

The GetMultiplePIDs methods are similar to GetModePID except that it can request up to six PIDs at once. This method will be used for vehicles communicating via the ISO 15765-4 protocol during dynamic data collection unless the BAR-OIS Software has determined that the vehicle will not properly respond to the GetMultiplePIDs method.

The following Mode/PIDs may be collected using the GetMultiplePIDs method:

Mode	PIDs
------	------

\$01	\$00-\$05, \$0A-\$54, \$59-\$FF (Fixed length PIDs only.)
------	---

Support PIDs will never be requested in the same command as data PIDs.

This method returns the following CommunicationResults:

Name
Success
VehicleTimedOut
DADError
VehicleError
DADNotConnected
VehicleNotConnected
NotSupported
InvalidData
DADTimedOut

3 UML

The following diagram details the IDAD interface to implement as UML.

